the image, converts to grayscale, converts from numpy to PyTorch tensor, and adds a batch dimension using the `.unsqueeze(dim=)` method. The `prepare_multi_state` function takes a tensor of dimensions `Batch x Channel x Height x Width` and updates the channel dimension with new frames. This function will only be used during the testing of the trained model; during training we will use a `deque` data structure to continuously append and pop frames. Lastly the `prepare_initial_state` function prepares the state when we first start the game and don't have a history of two prior frames. This function will copy the one frame three times to create a `Batch x 3 x Height x Width` tensor.

Listing 8.3. Preparing the states

```
import torch
from torch import nn
from torch import optim
import torch.nn.functional as F
from collections import deque

def prepare_state(state):                                    1
    return torch.from_numpy(downscale_obs(state,
to_gray=True)).float().unsqueeze(dim=0)


def prepare_multi_state(state1, state2):                     2
    state1 = state1.clone()
    tmp = torch.from_numpy(downscale_obs(state2,
to_gray=True)).float()
    state1[0][0] = state1[0][1]
    state1[0][1] = state1[0][2]
    state1[0][2] = tmp
    return state1


def prepare_initial_state(state,N=3):                        3
    state_ = torch.from_numpy(downscale_obs(state,
to_gray=True)).float()
    tmp = state_.repeat((N,1,1))
    return tmp.unsqueeze(dim=0)
```

- ***1*** Downscales state and converts to grayscale, converts to a PyTorch tensor, and finally adds a batch dimension